



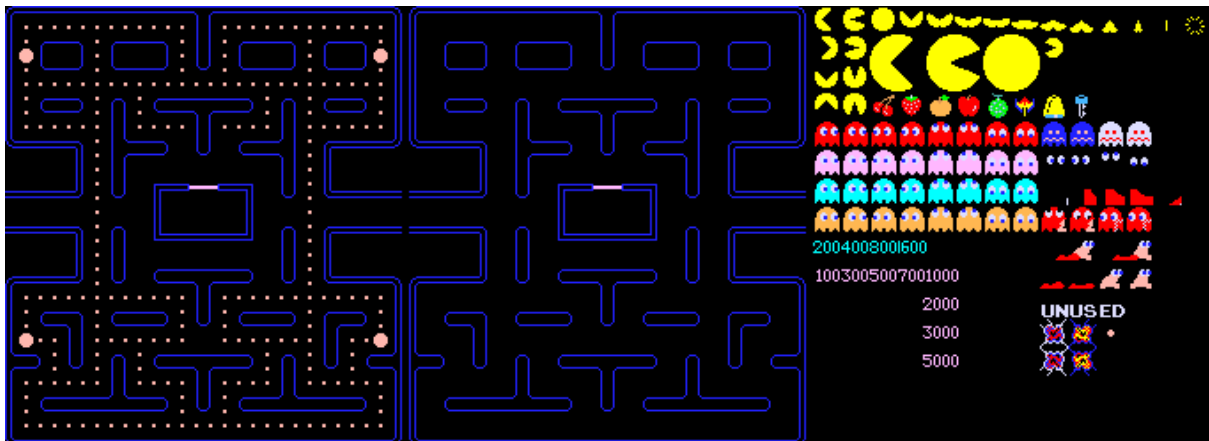
**UNIVERSIDAD NACIONAL AUTÓNOMA
DE MÉXICO
FACULTAD DE INGENIERÍA
2026-2
Fundamentos de programación
Trabajo Final**



Profesor: Ing. Adara Mercado Martínez

Equipo:

1. Aguilar Gonzales Ariadna Guadalupe 323291244
aguilargonzalezariadna@gmail.com
2. Limón Alegría Carlos Antonio 323096386
limoncarlos2007@gmail.com
3. Lugo Alvarez Xokami Janitzi 323304719 xokamilugo@gmail.com
4. Neri Galicia Leonardo 323206053
leonardonerigalicia2910@gmail.com



Introducción

El presente proyecto integrador tiene como objetivo aplicar los conocimientos adquiridos durante el curso de Fundamentos de Programación mediante el desarrollo de un videojuego tipo Pac-Man utilizando el lenguaje C y la biblioteca SDL2.

El proyecto permitirá integrar conceptos de:

- Resolución de problemas.
- Diagramas de flujo.
- Pseudocódigo, funciones.
- Arreglos unidimensionales y bidimensionales.
- Manejo de archivos.
- Modularización.
- Programación estructurada.

Objetivo General

Desarrollar un videojuego tipo Pac-Man simplificado en lenguaje C utilizando SDL2, aplicando programación estructurada y los conceptos fundamentales del curso.

Objetivos Específicos

El alumnado deberá:

- Diseñar soluciones mediante pseudocódigo y diagramas de flujo.
- Implementar funciones y modularización.
- Utilizar arreglos bidimensionales para representar mapas.
- Utilizar archivos para cargar niveles y guardar puntajes.
- Implementar lógica básica de videojuegos.
- Trabajar colaborativamente en un proyecto integrador.

Descripción General del Juego

El videojuego consiste en controlar un personaje dentro de un laberinto.

El jugador deberá:

- Recorrer el mapa.
- Consumir pellets.
- Evitar fantasmas.
- Acumular puntos.

El juego termina cuando:

- El jugador pierde todas sus vidas,
- O cuando se consumen todos los pellets del nivel.

Análisis del problema

Entradas:

- Archivo de configuración del mapa (**.txt**): Archivo externo de texto que contiene los caracteres (**#**, **.**, **P**, **F**, **)** que definen la estructura y el diseño del laberinto.
- Eventos del teclado (Usuario): Señales de entrada capturadas en tiempo real mediante SDL2 (Flechas de dirección: Arriba, Abajo, Izquierda, Derecha) para controlar el movimiento de Pac-Man.
- Archivo de puntajes (**.txt**): Archivo de texto que almacena el registro histórico de las puntuaciones más altas logradas por los jugadores.
- Teclas que se usarán para poder mover el jugador.
- Archivo con el mapa del juego.

Proceso:

1. Diseño del mapa

La estructura principal del proyecto consiste en una matriz bidimensional de caracteres de 10 filas por 10 columnas. Cada elemento de la matriz representa un objeto dentro del juego:

- '#' representa una pared.
- '.' representa un pellet.
- 'P' representa la posición de Pac-Man.
- 'F' representa la posición de un fantasma.

Esta representación permite gestionar de forma sencilla la lógica del juego y facilita la detección de colisiones y movimientos.

2. Sistema de renderizado

Para mostrar el contenido de la matriz en pantalla se utilizó SDL2. Cada celda de la matriz se convierte en un bloque gráfico de 40 × 40 píxeles. Dependiendo del carácter almacenado se asigna un color específico:

- Azul para las paredes.
- Blanco para los pellets.
- Amarillo para Pac-Man.
- Rojo para los fantasmas.

El proceso de renderizado se ejecuta continuamente dentro del ciclo principal del programa para actualizar la ventana gráfica.

3. Sistema de movimiento

El movimiento del jugador se realiza mediante eventos del teclado. Cuando el usuario presiona una flecha direccional, el programa calcula la nueva posición deseada dentro de la matriz.

Antes de efectuar el movimiento se verifica que la posición destino sea válida y que no corresponda a una pared. Si la casilla está libre, se actualiza la posición de Pac-Man dentro de la matriz.

4. Sistema de colisiones

La detección de colisiones se realiza verificando el contenido de la celda destino antes de mover al personaje.

Si la celda contiene una pared ('#'), el movimiento es cancelado. Si contiene un espacio libre o un pellet, el movimiento se permite y se actualiza la posición del jugador.

Salidas:

- Representación visual de todo el juego
- Puntaje acumulado
- Vidas restantes
- Avisar si hay derrota o victoria
- Puntaje final mostrado al terminar
- Ventana Gráfica (Interfaz de SDL2): Renderizado visual continuo del estado del juego en una ventana.
- Pantallas de Estado (Fin del Juego): Despliegue visual en pantalla que indica si el jugador obtuvo la Victoria (consumió todos los pellets) o la Derrota (perdió todas sus vidas).

Diagramas de flujo y pseudocódigo

Pseudocódigo:

Algoritmo PacmanUltraSimplificado

Definir mapa Como Caracter

Dimension mapa[5,5]

Definir pFila, pCol, nFila, nCol, i, j, puntaje Como Entero

Definir juegoActivo Como Logico

Definir tecla Como Caracter

```

// 1. INICIALIZAR MAPA (Paredes '#' en los bordes, pellets '.' dentro)
Para i <- 1 Hasta 5 Hacer
  Para j <- 1 Hasta 5 Hacer
    Si i=1 O i=5 O j=1 O j=5 Entonces
      mapa[i,j] <- "#"
    Sino
      mapa[i,j] <- "."
    FinSi
  FinPara
FinPara

// Posiciones iniciales fijas
mapa[2,4] <- "F" // Fantasma
pFila <- 2; pCol <- 2
mapa[pFila,pCol] <- "P" // Pac-Man

puntaje <- 0
juegoActivo <- Verdadero

// 2. CICLO PRINCIPAL
Mientras juegoActivo = Verdadero Hacer
  Borrar Pantalla
  Escribir "PUNTOS: ", puntaje

  // Dibujar mapa
  Para i <- 1 Hasta 5 Hacer
    Escribir mapa[i,1], " ", mapa[i,2], " ", mapa[i,3], " ", mapa[i,4], " ", mapa[i,5]
  FinPara

  // Leer Teclado
  Leer tecla
  nFila <- pFila; nCol <- pCol

  // Corrección de sintaxis en línea para PSeInt
  Si tecla="w" Entonces nFila <- pFila - 1; FinSi
  Si tecla="s" Entonces nFila <- pFila + 1; FinSi
  Si tecla="a" Entonces nCol <- pCol - 1; FinSi
  Si tecla="d" Entonces nCol <- pCol + 1; FinSi
  Si tecla="q" Entonces juegoActivo <- Falso; FinSi

// 3. COLISIONES Y MOVIMIENTO
Si mapa[nFila,nCol] = "#" Entonces
  Escribir "Pared"
  Esperar 1 Segundos

```

```

Sino
  Si mapa[nFila,nCol] = "F" Entonces
    Escribir "GAME OVER"
    juegoActivo <- Falso
  Sino
    Si mapa[nFila,nCol] = "." Entonces
      puntaje <- puntaje + 10
    FinSi
    // Mover en la matriz
    mapa[pFila,pCol] <- " "
    pFila <- nFila; pCol <- nCol
    mapa[pFila,pCol] <- "P"
  FinSi
FinSi
FinMientras
FinAlgoritmo

```

```

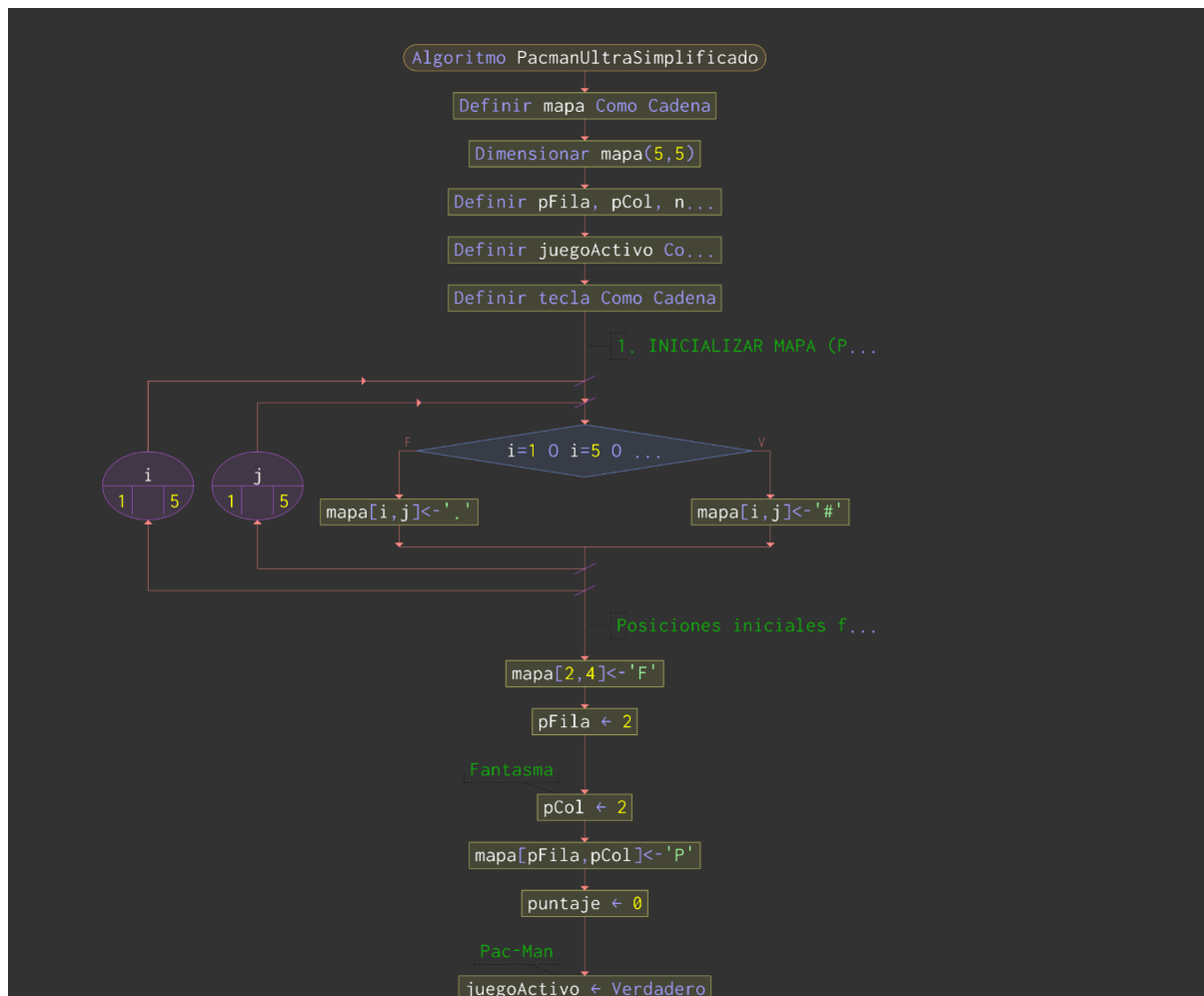
<sin_titulo>* <sin_titulo>* X
1  Algoritmo PacmanUltraSimplificado
2  Definir mapa Como Caracter
3  Dimension mapa[5,5]
4  Definir pFila, pCol, nFila, nCol, i, j, puntaje Como Entero
5  Definir juegoActivo Como Logico
6  Definir tecla Como Caracter
7
8  // 1. INICIALIZAR MAPA (Paredes '#' en los bordes, pellets '.' dentro)
9  Para i <- 1 Hasta 5 Hacer
10     Para j <- 1 Hasta 5 Hacer
11         Si i=1 O i=5 O j=1 O j=5 Entonces
12             mapa[i,j] <- "#"
13         Sino
14             mapa[i,j] <- "."
15         FinSi
16     FinPara
17 FinPara
18
19 // Posiciones iniciales fijas
20 mapa[2,4] <- "F" // Fantasma
21 pFila <- 2; pCol <- 2
22 mapa[pFila,pCol] <- "P" // Pac-Man
23
24 puntaje <- 0
25 juegoActivo <- Verdadero
26
27 // 2. CICLO PRINCIPAL
28 Mientras juegoActivo = Verdadero Hacer
29     Borrar Pantalla
30     Escribir "PUNTOS: ", puntaje
31
32     // Dibujar mapa
33     Para i <- 1 Hasta 5 Hacer
34         Escribir mapa[i,1], " ", mapa[i,2], " ", mapa[i,3], " ", mapa[i,4], " ", mapa[i,5]
35     FinPara
36
37     // Leer Teclado
38     Leer tecla
39     nFila <- pFila; nCol <- pCol
40
41     // Corrección de sintaxis en línea para PSeInt
42     Si tecla="w" Entonces nFila <- pFila - 1; FinSi
43     Si tecla="s" Entonces nFila <- pFila + 1; FinSi
44     Si tecla="a" Entonces nCol <- pCol - 1; FinSi
45     Si tecla="d" Entonces nCol <- pCol + 1; FinSi
46     Si tecla="q" Entonces juegoActivo <- Falso; FinSi

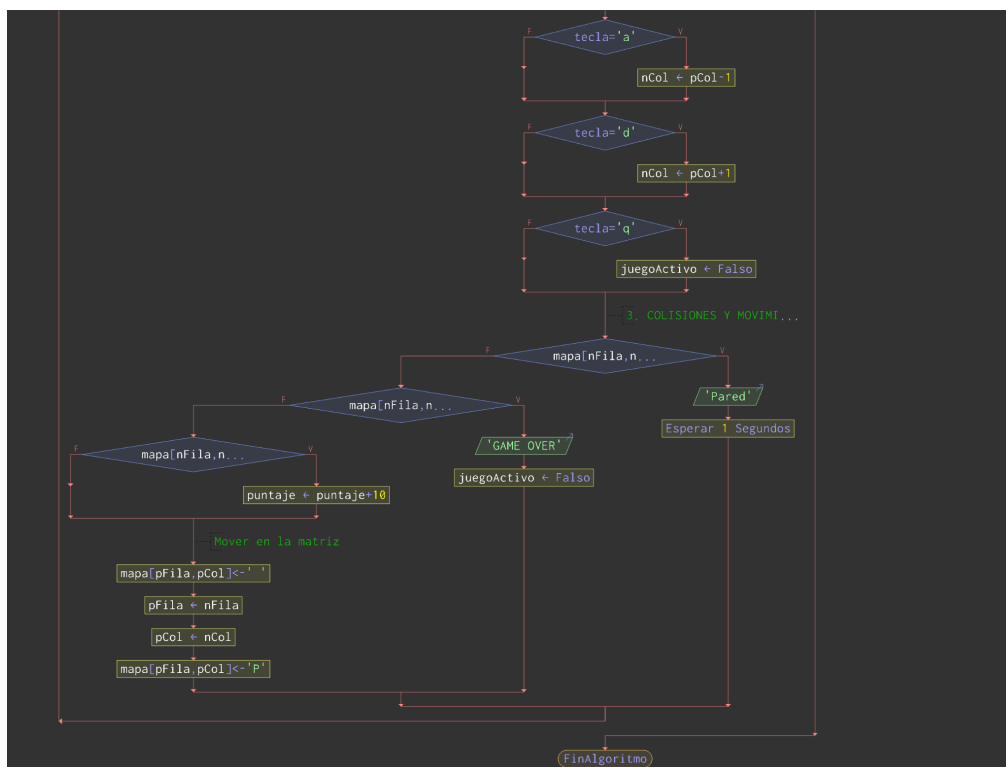
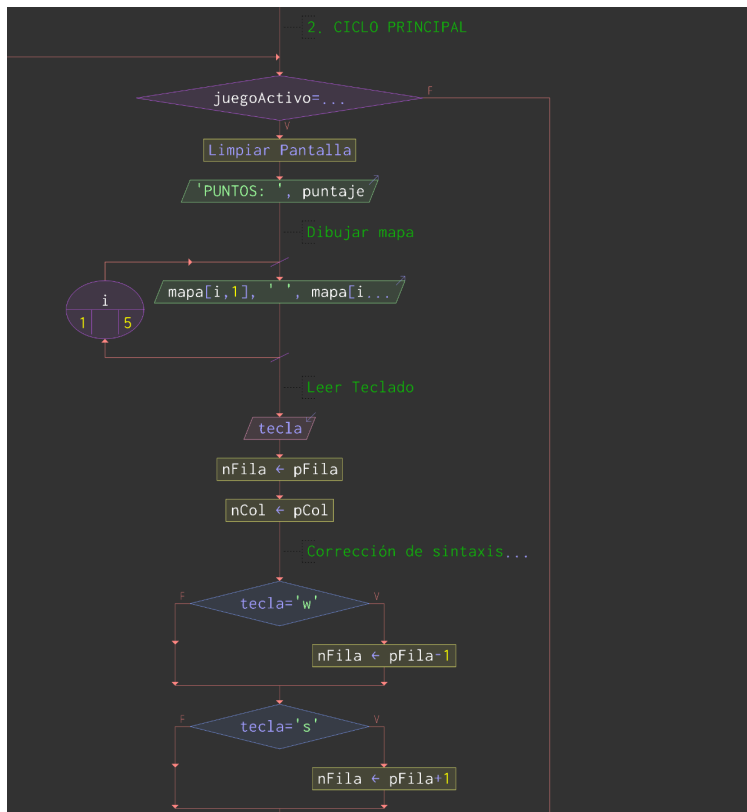
```

```

47
48 // 3. COLISIONES Y MOVIMIENTO
49 Si mapa[nFila,nCol] = "#" Entonces
50     Escribir "Pared"
51     Esperar 1 Segundos
52 Sino
53     Si mapa[nFila,nCol] = "F" Entonces
54         Escribir "GAME OVER"
55         juegoActivo ← Falso
56     Sino
57         Si mapa[nFila,nCol] = "." Entonces
58             puntaje ← puntaje + 10
59         FinSi
60         // Mover en la matriz
61         mapa[pFila,pCol] ← " "
62         pFila ← nFila; pCol ← nCol
63         mapa[pFila,pCol] ← "P"
64     FinSi
65 FinSi
66 FinMientras
67 FinAlgoritmo

```





//Para que pudiéramos entender todo de mejor manera lo que hicimos fue hacer dos códigos.//

Algoritmo MoverFantasmaAuxiliar

```
Definir fFila Como Entero
Definir fCol Como Entero
Definir pFila Como Entero
Definir pCol Como Entero
Definir nFila Como Entero
Definir nCol Como Entero
Definir i Como Entero
Definir j Como Entero
Definir mapa Como Caracter
Dimension mapa[5,5]

Para i <- 1 Hasta 5 Hacer
  Para j <- 1 Hasta 5 Hacer
    Si i = 1 O i = 5 O j = 1 O j = 5 Entonces
      mapa[i,j] <- "#"
    Sino
      mapa[i,j] <- "."
    FinSi
  FinPara
FinPara

fFila <- 3
fCol <- 3
pFila <- 2
pCol <- 4

mapa[fFila,fCol] <- "F"
mapa[pFila,pCol] <- "P"

nFila <- fFila
nCol <- fCol

Si pCol < fCol Entonces
  nCol <- fCol - 1
Sino
  Si pCol > fCol Entonces
    nCol <- fCol + 1
  FinSi
FinSi
```

Si pFila < fFila Entonces

 nFila <- fFila - 1

Sino

 Si pFila > fFila Entonces

 nFila <- fFila + 1

 FinSi

FinSi

Si mapa[nFila,nCol] = "#" Entonces

 Escribir "El fantasma detectó pared en camino. Cambia de dirección."

Sino

 // Mover físicamente en la matriz

 mapa[fFila,fCol] <- " "

 fFila <- nFila

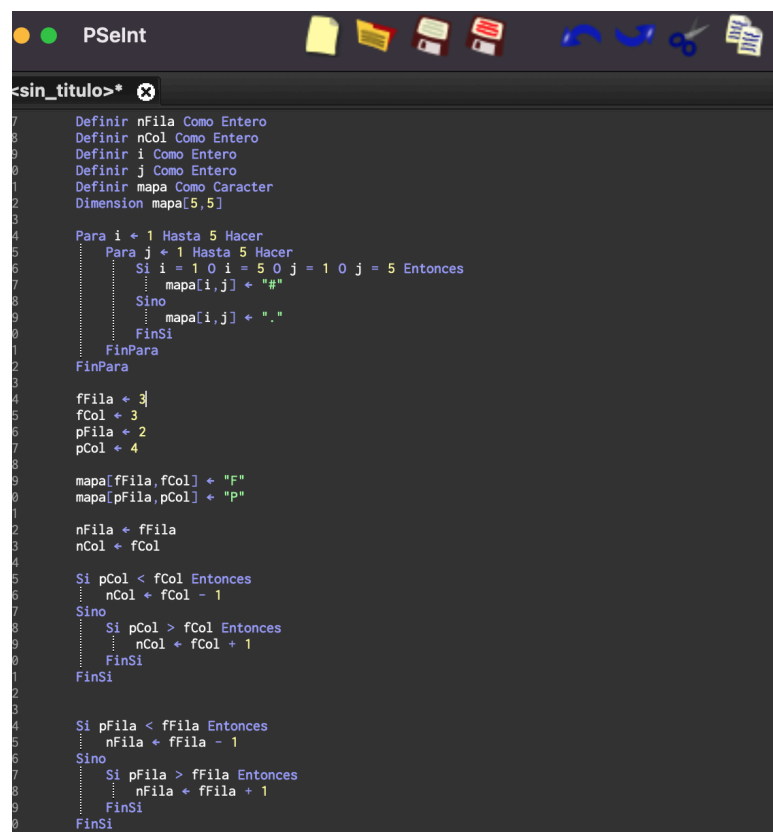
 fCol <- nCol

 mapa[fFila,fCol] <- "F"

 Escribir "Fantasma movido exitosamente a la casilla: ", fFila, ", ", fCol

FinSi

FinAlgoritmo

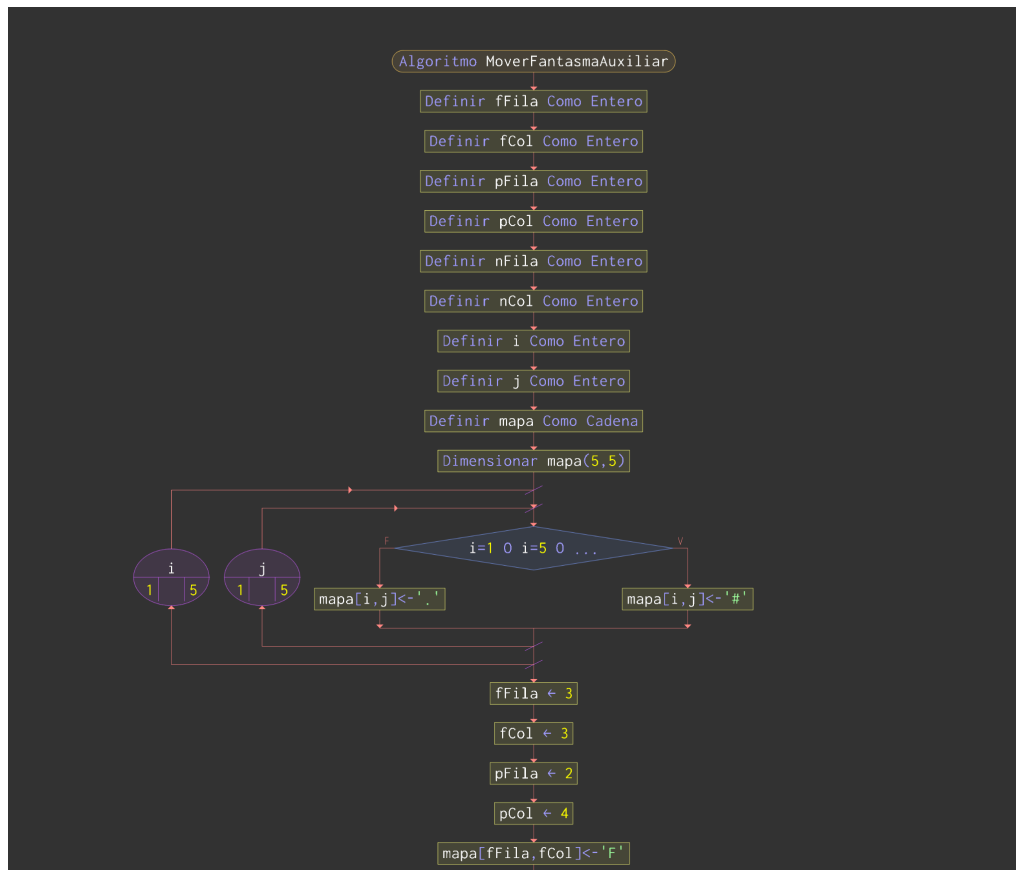


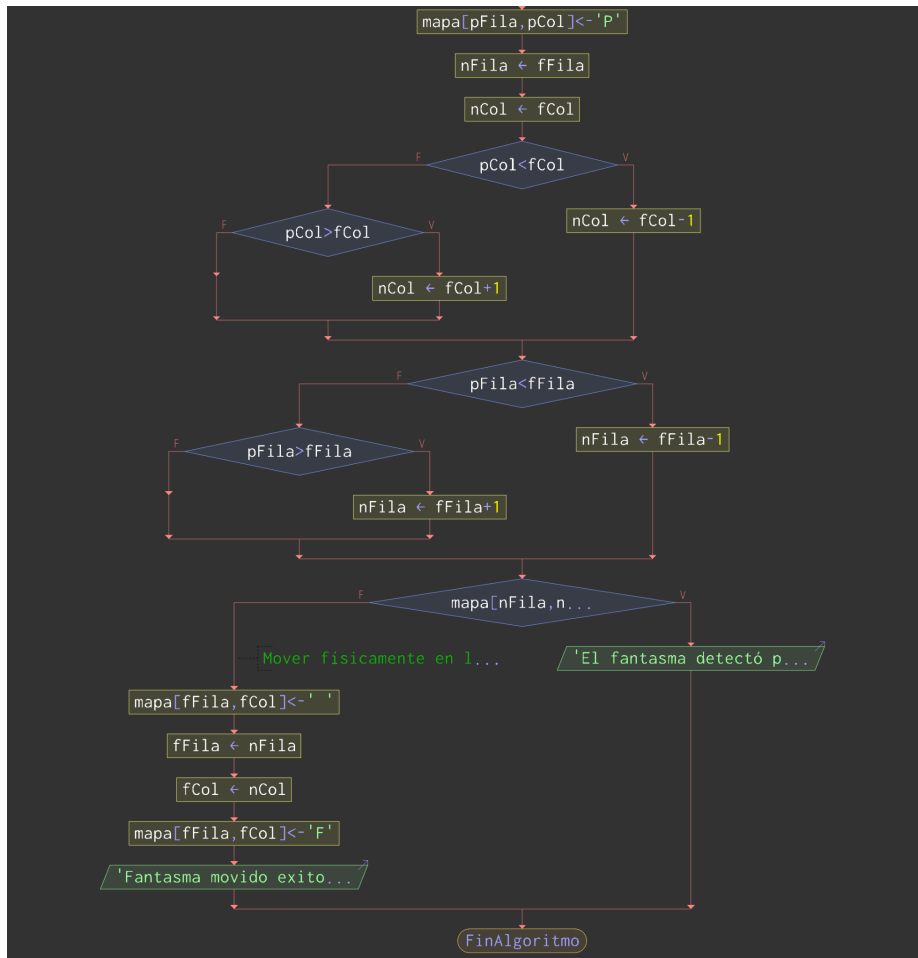
```
sin_titulo>*
7   Definir nFila Como Entero
8   Definir nCol Como Entero
9   Definir i Como Entero
10  Definir j Como Entero
11  Definir mapa Como Caracter
12  Dimension mapa[5,5]
13
14  Para i <- 1 Hasta 5 Hacer
15      Para j <- 1 Hasta 5 Hacer
16          Si i = 1 O i = 5 O j = 1 O j = 5 Entonces
17              mapa[i,j] <- "#"
18          Sino
19              mapa[i,j] <- "."
20          FinSi
21      FinPara
22  FinPara
23
24  fFila <- 3
25  fCol <- 3
26  pFila <- 2
27  pCol <- 4
28
29  mapa[fFila,fCol] <- "F"
30  mapa[pFila,pCol] <- "p"
31
32  nFila <- fFila
33  nCol <- fCol
34
35  Si pCol < fCol Entonces
36      nCol <- fCol - 1
37  Sino
38      Si pCol > fCol Entonces
39          nCol <- fCol + 1
40      FinSi
41  FinSi
42
43  Si pFila < fFila Entonces
44      nFila <- fFila - 1
45  Sino
46      Si pFila > fFila Entonces
47          nFila <- fFila + 1
48      FinSi
49  FinSi
```

```

51
52 Si mapa[nFila,nCol] = "#" Entonces
53     Escribir "El fantasma detectó pared en camino. Cambia de dirección."
54 Sino
55     // Mover físicamente en la matriz
56     mapa[ffila,fCol] ← " "
57     fFila ← nFila
58     fCol ← nCol
59     mapa[ffila,fCol] ← "F"
60     Escribir "Fantasma movido exitosamente a la casilla: ", fFila, ", ", fCol
61 FinSi
62
63 FinAlgoritmo

```



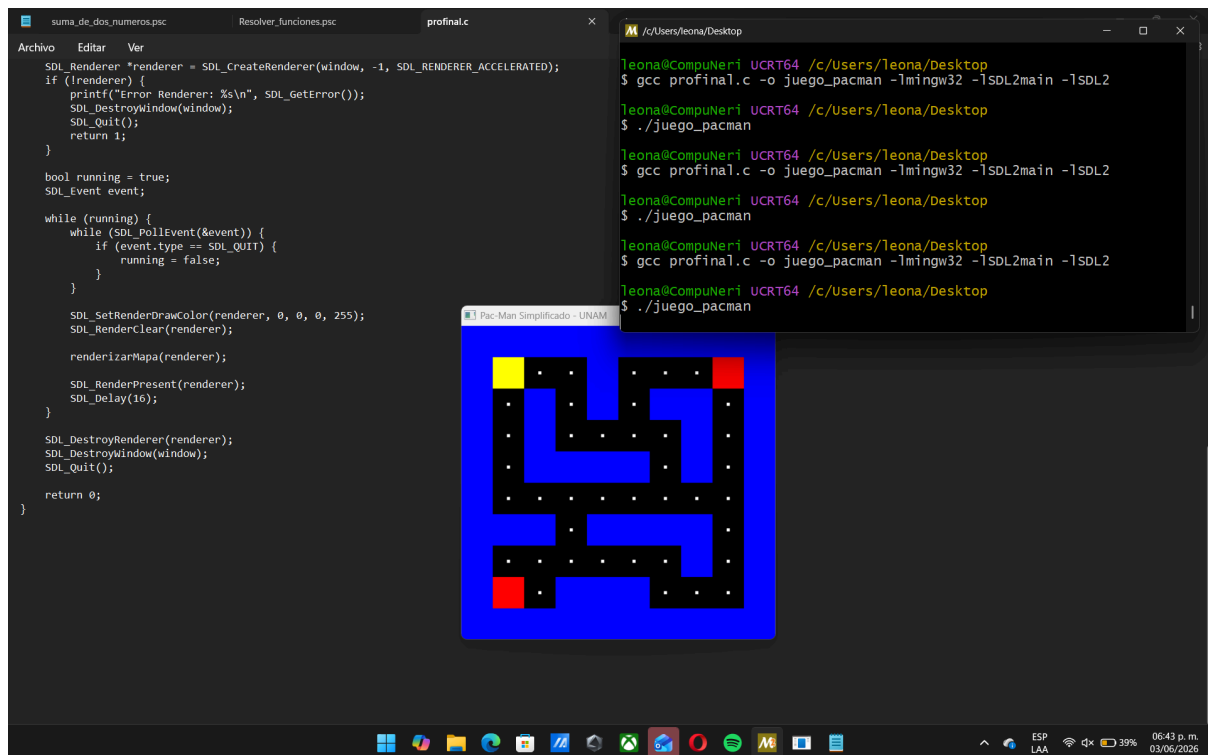


Una vez que logramos esquematizar de manera resumida y sencilla lo que debíamos de hacer comenzamos con el código, esta vez ya programando en c, para ello tuvimos que instalar un compilador y ocuparemos una biblioteca específica también instalamos “GIT”:

Para este punto empezamos con nuestro primer gran desafío, estábamos trabajando en la versión web de GitHub Codespaces. Intentamos configurar comandos locales ahí, pero nos dimos cuenta de que Codespaces no tiene una pantalla física nativa en tu navegador para abrir la ventana del juego que genera SDL2.

//El retroceso: Tuvimos que parar el trabajo en la nube y cambiar de estrategia para descargar el archivo profinal.c y compilarlo de forma nativa directamente en tu computadora para que la ventana pudiera abrirse en tu monitor.

Lo primero que compilamos es la base sólida de la estructura de datos es el arreglo bidimensional (La Matriz de caracteres) primera visualización del mapa.



```
#define SDL_MAIN_HANDLED
#include <SDL2/SDL.h>
#include <stdio.h>
#include <stdbool.h>
```

```
#define FILAS 10
#define COLUMNAS 10
#define TAMANIO_BLOQUE 40
```

```
char mapa[FILAS][COLUMNAS] = {
    {'#', '#', '#', '#', '#', '#', '#', '#', '#', '#'},
    {'#', 'P', '.', '.', '#', '.', '.', '.', 'F', '#'},
    {'#', '.', '#', '.', '#', '.', '#', '#', '.', '#'},
    {'#', '.', '#', '.', '.', '.', '.', '#', '.', '#'},
    {'#', '.', '#', '#', '#', '#', '.', '#', '.', '#'},
    {'#', '.', '.', '.', '.', '.', '.', '.', '.', '#'},
    {'#', '#', '#', '.', '#', '#', '#', '#', '.', '#'},
    {'#', '.', '.', '.', '.', '.', '.', '#', '.', '#'},
    {'#', 'F', '.', '#', '#', '#', '.', '.', '.', '#'},
    {'#', '#', '#', '#', '#', '#', '#', '#', '#', '#'}
};
```

```
void renderizarMapa(SDL_Renderer *renderer) {
    for (int i = 0; i < FILAS; i++) {
        for (int j = 0; j < COLUMNAS; j++) {
```

```

        SDL_Rect bloque = {
            j * TAMANIO_BLOQUE,
            i * TAMANIO_BLOQUE,
            TAMANIO_BLOQUE,
            TAMANIO_BLOQUE
        };

        if (mapa[i][j] == '#') {
            SDL_SetRenderDrawColor(renderer, 0, 0, 255, 255); // Paredes: Azul
            SDL_RenderFillRect(renderer, &bloque);
        }
        else if (mapa[i][j] == '.') {
            SDL_SetRenderDrawColor(renderer, 255, 255, 255, 255); // Pellets:
Blanco
            SDL_Rect pellet = {
                (j * TAMANIO_BLOQUE) + 18,
                (i * TAMANIO_BLOQUE) + 18,
                4,
                4
            };
            SDL_RenderFillRect(renderer, &pellet);
        }
        else if (mapa[i][j] == 'P') {
            SDL_SetRenderDrawColor(renderer, 255, 255, 0, 255); // Pac-Man:
Amarillo
            SDL_RenderFillRect(renderer, &bloque);
        }
        else if (mapa[i][j] == 'F') {
            SDL_SetRenderDrawColor(renderer, 255, 0, 0, 255); // Fantasma: Rojo
            SDL_RenderFillRect(renderer, &bloque);
        }
    }
}

int main(int argc, char **argv) {
    if (SDL_Init(SDL_INIT_VIDEO) != 0) {
        printf("Error %s\n", SDL_GetError());
        return 1;
    }

    SDL_Window *window = SDL_CreateWindow(
        "Pac-Man Simplificado - UNAM",
        SDL_WINDOWPOS_CENTERED,

```

```

        SDL_WINDOWPOS_CENTERED,
        COLUMNAS * TAMANIO_BLOQUE,
        FILAS * TAMANIO_BLOQUE,
        0
    );

    if (!window) {
        printf("Error: %s\n", SDL_GetError());
        SDL_Quit();
        return 1;
    }

    SDL_Renderer *renderer = SDL_CreateRenderer(window, -1,
    SDL_RENDERER_ACCELERATED);
    if (!renderer) {
        printf("Error Renderer: %s\n", SDL_GetError());
        SDL_DestroyWindow(window);
        SDL_Quit();
        return 1;
    }

    bool running = true;
    SDL_Event event;

    while (running) {
        while (SDL_PollEvent(&event)) {
            if (event.type == SDL_QUIT) {
                running = false;
            }
        }
    }

    SDL_SetRenderDrawColor(renderer, 0, 0, 0, 255);
    SDL_RenderClear(renderer);

    renderizarMapa(renderer);

    SDL_RenderPresent(renderer);
    SDL_Delay(16);
}

SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();

```

```
    return 0;
}
```

Además del movimiento del jugador, se implementó un sistema básico de inteligencia para los fantasmas. Para que puedan desplazarse automáticamente dentro del laberinto intentando acercarse a la posición actual de Pac-Man.

```
#define SDL_MAIN_HANDLED
#include <SDL2/SDL.h>
#include <stdio.h>
#include <stdbool.h>
```

```
// Dimensiones de la ventana y de las baldosas (tiles)
const int ANCHO_VENTANA = 640;
const int ALTO_VENTANA = 480;
const int TAMANO_TILE = 40; // Cada cuadro del mapa mide 40x40 píxeles
```

```
// Dimensiones del mapa (Arreglo bidimensional)
#define FILAS 10
#define COLUMNAS 15
```

```
// Variables globales para SDL2
SDL_Window* ventana = NULL;
SDL_Renderer* renderizador = NULL;
```

// REQUISITO 5.2 y 5.3: Representación del mapa ampliado

```
char mapa[FILAS][COLUMNAS] = {
    {'#','#','#','#','#','#','#','#','#','#','#','#','#'},
    {'#','.', '.', '.', '.', '.', '#', '.', '.', '.', '.', '.', '#'},
    {'#','.', '#','#', '.', '#', '.', '#', '.', '#', '.', '#', '.', '#'},
    {'#','.', '#', '.', '.', '#', '.', '.', '#', '.', '.', '#', '.', '#'},
    {'#','.', '.', '.', '#','#','#','#','#','#','#', '.', '.', '#'},
    {'#','.', '#', '.', '.', '.', '.', '.', '.', '#', '.', '#'},
    {'#','.', '#','#', '.', '#', '.', '#', '.', '#', '#', '.', '#'},
    {'#','.', '.', '.', '#', '.', '#', '.', '#', '.', '.', '#'},
    {'#','.', '.', '.', '.', '.', '.', '.', '.', '.', '#'},
    {'#','#','#','#','#','#','#','#','#','#','#','#','#'}
};
```

```
// Posiciones en la matriz (No en píxeles directos para facilitar colisiones después)
int pacmanX = 1;
int pacmanY = 1;
```

```
int fantasmaX = 13;
```



```

int fantasmaY = 8;

// Control de tiempo para el movimiento del fantasma
Uint32 ultimoMovimientoFantasma = 0;
const int RETARDO_FANTASMA = 300; // El fantasma se mueve cada 300
milisegundos

bool inicializarSDL() {
    if (SDL_Init(SDL_INIT_VIDEO) < 0) {
        printf("Error al inicializar SDL: %s\n", SDL_GetError());
        return false;
    }

    ventana = SDL_CreateWindow(
        "Pac-Man - Mapa Ampliado y Fantasma",
        SDL_WINDOWPOS_CENTERED,
        SDL_WINDOWPOS_CENTERED,
        ANCHO_VENTANA, ALTO_VENTANA,
        SDL_WINDOW_SHOWN
    );

    if (ventana == NULL) {
        return false;
    }

    renderizador = SDL_CreateRenderer(ventana, -1,
    SDL_RENDERER_ACCELERATED);
    if (renderizador == NULL) {
        return false;
    }

    return true;
}

void cerrarSDL() {
    SDL_DestroyRenderer(renderizador);
    SDL_DestroyWindow(ventana);
    SDL_Quit();
}

// REQUISITO 7.4: Función para renderizar el mapa, Pac-Man y el Fantasma
void renderizarTodo() {
    // Fondo negro
    SDL_SetRenderDrawColor(renderizador, 0, 0, 0, 255);

```

```

SDL_RenderClear(renderizador);

// Dibujar el mapa desde la matriz
for (int f = 0; f < FILAS; f++) {
    for (int c = 0; c < COLUMNAS; c++) {
        SDL_Rect tile = { c * TAMANO_TILE, f * TAMANO_TILE, TAMANO_TILE,
TAMANO_TILE };

        if (mapa[f][c] == '#') {
            SDL_SetRenderDrawColor(renderizador, 0, 0, 255, 255); // Paredes
Azules
            SDL_RenderFillRect(renderizador, &tile);
        } else if (mapa[f][c] == '.') {
            // Dibujar pellets pequeños en el centro de la baldosa
            SDL_SetRenderDrawColor(renderizador, 255, 255, 255, 255); // Blanco
            SDL_Rect pellet = { (c * TAMANO_TILE) + 18, (f * TAMANO_TILE) + 18,
4, 4 };
            SDL_RenderFillRect(renderizador, &pellet);
        }
    }
}

// Dibujar a Pac-Man (Cuadro Amarillo)
SDL_SetRenderDrawColor(renderizador, 255, 255, 0, 255);
SDL_Rect rectPacman = { pacmanX * TAMANO_TILE, pacmanY *
TAMANO_TILE, TAMANO_TILE, TAMANO_TILE };
SDL_RenderFillRect(renderizador, &rectPacman);

// Dibujar al Fantasma (Cuadro Rojo)
SDL_SetRenderDrawColor(renderizador, 255, 0, 0, 255);
SDL_Rect rectFantasma = { fantasmaX * TAMANO_TILE, fantasmaY *
TAMANO_TILE, TAMANO_TILE, TAMANO_TILE };
SDL_RenderFillRect(renderizador, &rectFantasma);

SDL_RenderPresent(renderizador);
}

int main(int argc, char* args[]) {
    if (!inicializarSDL()) {
        return 1;
    }

    bool juegoCorriendo = true;
    SDL_Event evento;

```

```

while (juegoCorriendo) {

    // 1. CAPTURA DE EVENTOS (Movimiento de Pac-Man por celdas)
    while (SDL_PollEvent(&evento) != 0) {
        if (evento.type == SDL_QUIT) {
            juegoCorriendo = false;
        }
        else if (evento.type == SDL_KEYDOWN) {
            int siguienteX = pacmanX;
            int siguienteY = pacmanY;

            switch (evento.key.keysym.sym) {
                case SDLK_UP:    siguienteY--; break;
                case SDLK_DOWN:  siguienteY++; break;
                case SDLK_LEFT:  siguienteX--; break;
                case SDLK_RIGHT: siguienteX++; break;
            }

            // REQUISITO 5.4: Detección básica de colisiones con paredes para
Pac-Man
            if (mapa[siguienteY][siguienteX] != '#') {
                pacmanX = siguienteX;
                pacmanY = siguienteY;

                // Si pasa sobre un pellet, se lo come (lo borra de la matriz)
                if (mapa[pacmanY][pacmanX] == '.') {
                    mapa[pacmanY][pacmanX] = ' ';
                }
            }
        }
    }

    // 2. LÓGICA DEL FANTASMA (Movimiento automático por tiempo)
    Uint32 tiempoActual = SDL_GetTicks();
    if (tiempoActual - ultimoMovimientoFantasma > RETARDO_FANTASMA) {
        int sigFantasmaX = fantasmaX;
        int sigFantasmaY = fantasmaY;

        // REQUISITO 5.5.2: Comportamiento sugerido (Persecución simple en X e
Y)
        if (pacmanX < fantasmaX) sigFantasmaX--;
        else if (pacmanX > fantasmaX) sigFantasmaX++;
        else if (pacmanY < fantasmaY) sigFantasmaY--;
    }
}

```

```

        else if (pacmanY > fantasmaY) sigFantasmaY++;

        // Si el movimiento no choca con pared, el fantasma avanza
        if (mapa[sigFantasmaY][sigFantasmaX] != '#') {
            fantasmaX = sigFantasmaX;
            fantasmaY = sigFantasmaY;
        }

        ultimoMovimientoFantasma = tiempoActual;
    }

    // REQUISITO 5.7: Detección de colisión entre Pac-Man y el Fantasma (Fin del
    juego preliminar)
    if (pacmanX == fantasmaX && pacmanY == fantasmaY) {
        printf("¡El fantasma te atrapo! Fin de la partida.\n");
        juegoCorriendo = false;
    }

    // 3. RENDERIZADO
    renderizarTodo();

    SDL_Delay(16);
}

cerrarSDL();
return 0;
}

```

1. Abstracción del Laberinto como Matriz Estática (Líneas 1-22)

La declaración directivas del preprocesador `#define` para establecer las dimensiones fijas del escenario (**FILAS 10**, **COLUMNAS 10**) y la escala de conversión de coordenadas lógicas a píxeles en pantalla (**TAMANIO_BLOQUE 40**).

Posteriormente, se inicializó explícitamente un arreglo bidimensional de tipo `char` se llamado `mapa[FILAS][COLUMNAS]`. Este es un paso fundamental del juego ya que esta matriz de caracteres. En lugar de trabajar con coordenadas flotantes o continuas, el laberinto se discretiza en una cuadrícula rígida. Se definió un diccionario de símbolos estándar: `#` representa los límites físicos impenetrables (paredes), `.` representa las celdas con puntaje (pellets), `P` localiza la posición de inicio del jugador, y `F` sitúa a los enemigos (fantasmas). Esta estructura permite un acceso directo en complejidad constante $O(1)$ a cualquier coordenada espacial

mediante los índices del arreglo.

PDF

2. Función de Traducción Gráfica: **renderizarMapa**

Se implementó una función modular que recibe el puntero del contexto de renderizado de SDL2 (**SDL_Renderer *renderer**). Utiliza dos bucles anidados (**for**) que recorren exhaustivamente cada casilla de la matriz desde la posición (0,0) hasta la (9,9). Por cada iteración, la función calcula dinámicamente un contenedor geométrico (**SDL_Rect bloque**) multiplicando los índices lógicos **i** (fila) y **j** (columna) por el tamaño en píxeles del bloque (**TAMANIO_BLOQUE**). Dentro de los ciclos, una estructura de decisión selectiva múltiple (**if-else if**) evalúa el carácter almacenado en la celda actual:

Posteriormente se crearon dos variables globales enteras (**pacmanFila** y **pacmanColumna**) para almacenar la ubicación exacta del jugador en todo momento. Se diseñó la función auxiliar **buscarPacman()**, la cual se ejecuta **una sola vez** justo antes de iniciar el ciclo principal del juego. Se desarrolló la lógica principal de interacción física del juego mediante la función **moverJugador(int nuevaFila, int nuevaColumna)**, la cual opera bajo tres fases estrictas de validación de programación estructurada:

Dentro del ciclo continuo del juego (**while (running)**), se integró el despachador de eventos de SDL2 (**SDL_PollEvent**). Se configuró un evaluador de tipo de evento

Finalmente, en cada ciclo del bucle, el renderizador limpia el marco anterior con fondo negro (**SDL_RenderClear**), dibuja el estado actualizado de la matriz mediante **renderizarMapa(renderer)** y proyecta el resultado final en la pantalla del usuario actualizándose de forma constante cada 16 milisegundos (**SDL_Delay(16)**) para simular una tasa de refresco fluida de aproximadamente 60 fotogramas por segundo (FPS).

1. El Concepto Lógico del Movimiento en la Matriz

En lugar de movernos por píxeles libres, nos moveremos celda por celda dentro del arreglo bidimensional. Cuando presionas una tecla, calculamos la **posición destino** (la casilla a la que Pac-Man quiere ir):

- **Flecha Arriba:** Fila destino = **filaActual - 1**
- **Flecha Abajo:** Fila destino = **filaActual + 1**
- **Flecha Izquierda:** Columna destino = **columnaActual - 1**
- **Flecha Derecha:** Columna destino = **columnaActual + 1**

Antes de autorizar el movimiento, revisamos el valor de `mapa[Fila destino][Columna destino]`. Si es una pared (#), ignoramos la pulsación. Si es un camino libre o un pellet (.), actualizamos la matriz.

```
#define SDL_MAIN_HANDLED
#include <SDL2/SDL.h>
#include <stdio.h>
#include <stdbool.h>

#define FILAS 10
#define COLUMNAS 10
#define TAMANIO_BLOQUE 40

char mapa[FILAS][COLUMNAS] = {
    {'#', '#', '#', '#', '#', '#', '#', '#', '#', '#'},
    {'#', 'P', '.', '.', '#', '.', '.', '.', 'F', '#'},
    {'#', '.', '#', '.', '#', '.', '#', '#', '.', '#'},
    {'#', '.', '#', '.', '.', '.', '.', '#', '.', '#'},
    {'#', '.', '#', '#', '#', '#', '.', '#', '.', '#'},
    {'#', '.', '.', '.', '.', '.', '.', '.', '.', '#'},
    {'#', '#', '#', '.', '#', '#', '#', '#', '.', '#'},
    {'#', '.', '.', '.', '.', '.', '.', '#', '.', '#'},
    {'#', 'F', '.', '#', '#', '#', '.', '.', '.', '#'},
    {'#', '#', '#', '#', '#', '#', '#', '#', '#', '#'}
};
```

// VARIABLES GLOBALES PARA LA POSICIÓN DE PAC-MAN EN LA MATRIZ

```
int pacmanFila = 0;
int pacmanColumna = 0;
```

// REQUISITO 7.4: Función para localizar a Pac-Man ('P') al iniciar

```
void buscarPacman() {
    for (int i = 0; i < FILAS; i++) {
        for (int j = 0; j < COLUMNAS; j++) {
            if (mapa[i][j] == 'P') {
                pacmanFila = i;
                pacmanColumna = j;
                return; // Ya lo encontramos, salimos de la función
            }
        }
    }
}
```

// REQUISITO 5.4 y 7.4: Función para mover al jugador con colisiones

```

void moverJugador(int nuevaFila, int nuevaColumna) {
    // 1. Validar que no se salga de los límites de la matriz por seguridad
    if (nuevaFila >= 0 && nuevaFila < FILAS && nuevaColumna >= 0 &&
        nuevaColumna < COLUMNAS) {

        // 2. DETECTAR COLISIÓN: Si la casilla destino es una pared '#', no hacemos
nada
        if (mapa[nuevaFila][nuevaColumna] == '#') {
            return;
        }

        // 3. ACTUALIZAR MATRIZ: Si no es pared, Pac-Man se mueve
        // Borramos la 'P' de su lugar viejo y dejamos un camino libre vacío ' '
        mapa[pacmanFila][pacmanColumna] = ' ';

        // Actualizamos nuestras variables de rastreo con las nuevas coordenadas
        pacmanFila = nuevaFila;
        pacmanColumna = nuevaColumna;

        // Colocamos la 'P' en la nueva posición de la matriz
        mapa[pacmanFila][pacmanColumna] = 'P';
    }
}

void renderizarMapa(SDL_Renderer *renderer) {
    for (int i = 0; i < FILAS; i++) {
        for (int j = 0; j < COLUMNAS; j++) {

            SDL_Rect bloque = {
                j * TAMANIO_BLOQUE,
                i * TAMANIO_BLOQUE,
                TAMANIO_BLOQUE,
                TAMANIO_BLOQUE
            };

            if (mapa[i][j] == '#') {
                SDL_SetRenderDrawColor(renderer, 0, 0, 255, 255); // Paredes: Azul
                SDL_RenderFillRect(renderer, &bloque);
            }
            else if (mapa[i][j] == '.') {
                SDL_SetRenderDrawColor(renderer, 255, 255, 255, 255); // Pellets:
Blanco
                SDL_Rect pellet = {
                    (j * TAMANIO_BLOQUE) + 18,

```

```

        (i * TAMANIO_BLOQUE) + 18,
        4,
        4
    };
    SDL_RenderFillRect(renderer, &pellet);
}
else if (mapa[i][j] == 'P') {
    SDL_SetRenderDrawColor(renderer, 255, 255, 0, 255); // Pac-Man:
Amarillo
    SDL_RenderFillRect(renderer, &bloque);
}
else if (mapa[i][j] == 'F') {
    SDL_SetRenderDrawColor(renderer, 255, 0, 0, 255); // Fantasma: Rojo
    SDL_RenderFillRect(renderer, &bloque);
}
}
}
}

int main(int argc, char **argv) {
    if (SDL_Init(SDL_INIT_VIDEO) != 0) {
        printf("Error %s\n", SDL_GetError());
        return 1;
    }

    SDL_Window *window = SDL_CreateWindow(
        "Pac-Man - Movimiento y Colisiones",
        SDL_WINDOWPOS_CENTERED,
        SDL_WINDOWPOS_CENTERED,
        COLUMNAS * TAMANIO_BLOQUE,
        FILAS * TAMANIO_BLOQUE,
        0
    );

    if (!window) {
        SDL_Quit();
        return 1;
    }

    SDL_Renderer *renderer = SDL_CreateRenderer(window, -1,
    SDL_RENDERER_ACCELERATED);
    if (!renderer) {
        SDL_DestroyWindow(window);
        SDL_Quit();
    }
}

```



```

    return 1;
}

// ¡PASO CLAVE!: Encontrar dónde está Pac-Man antes de iniciar el ciclo
buscarPacman();

bool running = true;
SDL_Event event;

while (running) {
    // REQUISITO 5.1 y 5.4: Captura de eventos de teclado
    while (SDL_PollEvent(&event)) {
        if (event.type == SDL_QUIT) {
            running = false;
        }
        else if (event.type == SDL_KEYDOWN) {
            switch (event.key.keysym.sym) {
                case SDLK_UP:
                    moverJugador(pacmanFila - 1, pacmanColumna);
                    break;
                case SDLK_DOWN:
                    moverJugador(pacmanFila + 1, pacmanColumna);
                    break;
                case SDLK_LEFT:
                    moverJugador(pacmanFila, pacmanColumna - 1);
                    break;
                case SDLK_RIGHT:
                    moverJugador(pacmanFila, pacmanColumna + 1);
                    break;
            }
        }
    }
}

SDL_SetRenderDrawColor(renderer, 0, 0, 0, 255);
SDL_RenderClear(renderer);

renderizarMapa(renderer);

SDL_RenderPresent(renderer);
SDL_Delay(16);
}
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();

```

```
    return 0;
}
```

El programa desarrollado implementa un videojuego bidimensional interactivo en tiempo real utilizando un paradigma de **programación estructurada modular**. A nivel arquitectónico, el sistema se divide en tres capas conceptuales que operan en un ciclo continuo: *Captura de Eventos*, *Lógica de Control (Física y Colisiones)* y *Subsistema Gráfico*.

El comportamiento del software se puede descomponer mediante el estándar clásico de ingeniería:

A) Entradas (Inputs)

- **Eventos de Hardware (Teclado):** Señales asíncronas generadas por el usuario al oprimir las flechas de dirección periféricas (**SDLK_UP**, **SDLK_DOWN**, **SDLK_LEFT**, **SDLK_RIGHT**).
- **Señales del Sistema Operativo:** Eventos de control de ventana, específicamente la señal de terminación (**SDL_QUIT**) enviada al presionar el botón de cierre ("X").
- **Estructura Estática de Datos:** El arreglo bidimensional hardcodeado en la memoria que define la geometría inicial del laberinto (10×10).

B) Procesos (Process)

- **Sincronización de Estado (buscarPacman):** Escaneo secuencial lineal de complejidad temporal $O(N \times M)$ para indexar las variables de rastreo globales con la posición de inicio del personaje en el mapa.
- **Proyección de Coordenadas:** Conversión algebraica lineal instantánea en el bloque **switch-case** para determinar la casilla destino teórica a la que el usuario desea desplazarse.
- **Validación de Fronteras (Seguridad de Memoria):** Evaluación lógica booleana condicional para asegurar la invariabilidad de los índices dentro del rango dinámico permitido por el arreglo [0,9].
- **Algoritmo de Colisión Estática:** Evaluación selectiva del contenido de memoria en la celda destino. Interrupción por retorno anticipado (**return**) si se escanea un obstáculo ('#').
- **Actualización Atómica del Mapa:** Transición de estados en la memoria RAM que limpia la posición anterior con un espacio vacío y escribe el carácter 'P' en el nuevo índice aprobado.

C) Salidas (Outputs)

- **Renderizado de Primitivas Gráficas:** Conversión de la matriz lógica de caracteres en bloques geométricos sólidos coloreados (`SDL_Rect`) en una ventana física de 400×400 píxeles.
 - **Control de Tasa de Refresco:** Retardo síncrono forzado de 16 milisegundos (`SDL_Delay(16)`) para estabilizar la salida visual a ~60 FPS, evitando la sobresaturación del procesador.
 - **Logs de Consola:** Impresión de mensajes de diagnóstico en la salida estándar (`stdout`) mediante `printf` en caso de fallas críticas en los subsistemas de video.
1. **Uso de Variables Globales de Rastreo:** Almacenar `pacmanFila` y `pacmanColumna` de forma global evita la necesidad de escanear recursivamente las 100 celdas de la matriz en cada pulsación de tecla. Esto reduce la complejidad de la búsqueda de un costo lineal $O(N \times M)$ a un costo constante de acceso directo $O(1)$, optimizando el rendimiento general de la CPU.
 2. **Abstracción Gráfica Desacoplada:** La función `renderizarMapa` no sabe ni le importa si el jugador se está moviendo o si los fantasmas están persiguiendo; simplemente toma una "fotografía" del estado actual de la matriz de caracteres y la dibuja. Este desacoplamiento entre la lógica y los gráficos es una excelente práctica de ingeniería de software que facilita el mantenimiento futuro del código.

Bitácora

1. El Concepto Lógico del Movimiento en la Matriz
En lugar de movernos por píxeles libres, nos moveremos celda por celda dentro del arreglo bidimensional.
Cuando presionas una tecla, calculamos la posición destino (la casilla a la que Pac-Man quiere ir):

Es un excelente código de base para un clon de Pac-Man en 2D utilizando SDL2. Está muy bien estructurado, es limpio, fácil de leer y sigue una lógica clásica de desarrollo de videojuegos basada en rejillas (*tile-based movement*).

Aquí tienes un análisis detallado de lo que está muy bien hecho, junto con algunas sugerencias de mejora y correcciones técnicas para llevarlo al siguiente nivel:

🌟 Puntos Fuertes del Código

- **Lógica de Rejilla Impecable:** Separar el movimiento por celdas (`FILAS` y `COLUMNAS`) en lugar de píxeles libres es la mejor decisión para un juego tipo Pac-Man. Facilita enormemente el cálculo de colisiones.
- **Modularidad:** Las funciones `buscarPacman()`, `moverJugador()` y `renderizarMapa()` tienen una sola responsabilidad clara, lo cual es una gran práctica en programación en C.
- **Control de Desbordamiento:** En `moverJugador`, verificar que `nuevaFila` y `nuevaColumna` estén dentro de los límites de la matriz antes de evaluar el contenido evita que el programa intente leer memoria inválida (un clásico *segmentation fault*).
- **Limpieza de Recursos:** Al final de `main`, destruyes correctamente el `renderer`, la ventana y

🔧 Áreas de Mejora y Pequeños Detalles

1. El Dilema del Movimiento de Pac-Man

En el Pac-Man original, tú no presionas una tecla para avanzar una sola casilla. Presionas una dirección y Pac-Man **sigue avanzando automáticamente** en esa dirección hasta chocar con una pared.

- **Estado actual:** Tu código requiere que el usuario presione repetidamente las flechas para caminar (un toque por casilla).
- **Solución futura:** Podrías implementar una variable `direccionActual` que se actualice con el teclado, y mover a Pac-Man de forma automática usando un temporizador dentro del ciclo principal.

2. El "Parpadeo" del Renderizado de Pellets

En la función `renderizarMapa()`, cuando dibujas un pellet (.), primero evalúas si la matriz tiene ese carácter. Sin embargo, no estás limpiando el fondo de esa celda específica. Como tu fondo general es negro (`SDL_RenderClear`), funciona bien por ahora, pero si quisieras poner un fondo texturizado, verías problemas.

3. Las Constantes Mágicas (*Magic Numbers*)

En el dibujo del pellet tienes esto: